



## Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

|                                   |          |                                            |
|-----------------------------------|----------|--------------------------------------------|
| Desarrollo de<br>Aplicaciones Web | Unidad 2 | Tema: EL Patrón DTO (Data Transfer Object) |
|                                   | Semana 4 | Tutor: Ing. Victoria Castro                |

## ¿Qué es el Patrón DTO?

El **DTO** es un patrón de diseño cuyo objetivo es transportar datos entre procesos o capas de una aplicación (por ejemplo, de la Base de Datos al Controlador REST) para reducir el número de llamadas a métodos y, lo más importante, **desacoplar** la estructura interna de la base de datos de la respuesta que ve el cliente.

¿Por qué no usar directamente las "Entidades" (@Entity)?

**Seguridad:** Una entidad suele tener campos sensibles (password, roles internos, fechas de auditoría) que no deben enviarse al frontend.

**Optimización:** El cliente a veces solo necesita 3 campos de una tabla que tiene 50.

**Flexibilidad:** Si cambias el nombre de una columna en tu base de datos, no rompes el contrato con el frontend si usas un DTO como intermediario.

**Recursividad:** Evita errores de "referencia circular" (StackOverflowError) cuando hay relaciones @ManyToOne o @OneToMany.

## Ejemplo Práctico: Spring Boot 3.x y Java Records

En **Spring Boot 3** y versiones modernas de Java (17+), la mejor forma de implementar DTOs es mediante **Java Records**. Los Records son inmutables, concisos y eliminan el código repetitivo (Boilerplate) como getters y constructores.

### Escenario: Un sistema de usuarios

Queremos guardar un usuario con contraseña, pero al consultar sus datos, **nunca** debemos devolver la contraseña.

## A. La Entidad

```
@Entity
@Table(name = "usuarios")
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private String email;
    private String password; // Campo sensible

    // Getters y Setters...
}
```

## B. El DTO

Usamos un record para que sea una clase de solo datos, limpia y segura.

```
public record UsuarioResponseDTO(
    Long id,
    String nombre,
    String email
) {}
```

## C. El Controlador

Aquí transformamos la entidad en DTO antes de enviarla por HTTP.

```
@RestController
@RequestMapping("/api/usuarios")
public class UsuarioController {

    @GetMapping("/{id}")
    public ResponseEntity<UsuarioResponseDTO> obtenerUsuario(@PathVariable Long id) {
        // Simulamos obtener la entidad de la DB
        Usuario usuario = usuarioService.buscarPorId(id);

        // Mapeamos la Entidad al DTO
        UsuarioResponseDTO dto = new UsuarioResponseDTO(
            usuario.getId(),
            usuario.getNombre(),
            usuario.getEmail()
        );

        return ResponseEntity.ok(dto);
    }
}
```

## Material Visual Complementario

Para ver una implementación paso a paso y entender cómo este patrón ayuda a limpiar la arquitectura de tus proyectos, consulta este recurso:

- **Video Recomendado:** [Explicación del Patrón DTO y su implementación](#)

